

Bluetooth Slave Mode

Bluetooth Slave Mode

1. BLE Overview
2. Core Concepts
 - 2.1 GAP — Generic Access Profile
 - 2.2 GATT — Generic Attribute Profile
 - 2.3 Notify and Write
 - 2.4 NimBLE Protocol Stack
3. ESP32-S3 BLE Hardware Features
4. Project Architecture and Flow
5. Hardware Dependencies
6. Source Code Analysis
 - 6.1 Header Files and Macro Configuration
 - 6.2 app_main() — Entry Point
 - 6.3 ble_slave_init() — Initialization
 - 6.4 on_sync() — Sync Callback and start_advertising()
 - 6.5 gap_event_cb() — GAP Event Callback
 - 6.6 GATT Service Definition — Service Table Structure
 - 6.7 gatt_svr_access_cb() — Write Buffer Accumulation
 - 6.8 notify_data() — Notification Send
 - 6.9 vNotifyTask() — Periodic Notification Task
7. Operation Flow
 - 7.1 Build and Flash Flow
 - 7.2 Testing Method 1: With ESP32 Master
 - 7.3 Testing Method 2: With Phone Bluetooth Debugging Assistant
 - 7.4 Normal Communication Log (Dual-Board Test)
 - 7.5 Edge Cases
8. Common Problems

1. BLE Overview

BLE (Bluetooth Low Energy) is a short-range wireless communication protocol designed for low power consumption and low bandwidth scenarios. Compared to classic Bluetooth, BLE connects faster (millisecond level) and consumes less power (coin battery can run for months).

Importance of BLE Slave tutorial: BLE slave (Peripheral) is the most common role form for IoT devices—sensor nodes, wearable devices, smart home peripherals, etc. Most scenarios run as slave. Mastering slave programming means being able to build smart devices discoverable and connectable by phones, tablets, or gateways—this is the core capability of embedded BLE development.

Typical application scenarios: Temperature/humidity sensors periodically report ambient data, smart door locks receive unlock commands from phones, heart rate belts continuously push health data, electronic tags receive price updates.

BLE role system:

Role	GAP Name	Responsibilities
Slave	Peripheral	Advertises its presence, accepts connections, provides GATT service for master access
Master	Central	Actively scans nearby devices, initiates connections, discovers services and characteristics, reads/writes data

This project implements **BLE Peripheral (Slave)**, advertises with device name "ESP-IDF", provides custom GATT service 0xFFFF0, supports notification reporting (0xFFFF1) and write receiving (0xFFFF2).

2. Core Concepts

2.1 GAP — Generic Access Profile

GAP (Generic Access Profile) defines how BLE devices **are scanned and connected**.

GAP operations involved in this slave:

Operation	API	Description
Advertising	ble_gap_adv_start()	Periodically sends advertising packets, containing device name and service UUID
Connection response	gap_event_cb()	Accepts master connection request, updates connection handle
Disconnection re-advertise	gap_event_cb()	Detects disconnection then automatically restarts advertising

Advertising data composition:

```
Advertising Packet
├─ Flags: BLE_HS_ADV_F_DISC_GEN | BLE_HS_ADV_F_BREDR_UNSUP
│   General discoverable mode + Classic Bluetooth not supported
├─ Name: "ESP-IDF" (Complete)
│   Complete device name, master filters by this
└─ Service UUID: 0xFFFF0 (Complete)
    Custom service, master can filter devices by this UUID during scan
```

2.2 GATT — Generic Attribute Profile

GATT (Generic Attribute Profile) defines how BLE devices **organize data**.

This project's custom GATT service architecture:

```
Service: 0xFFFF0 (Custom primary service)
├─ Characteristic: 0xFFFF1
│   Property: NOTIFY (slave proactively notifies master, no master request needed)
│   Direction: Slave → Master
│   Content: "Hello from ESP32 Slave [counter]"
│
└─ Characteristic: 0xFFFF2
    Property: WRITE (receives data sent by master)
    Direction: Master → Slave
    Content: "Hello from ESP32 Master"
```

2.3 Notify and Write

Operation	Initiator	Target	Confirmed	Typical Scenario
Notify	Slave → Master	0xFFFF1	No confirmation needed	Sensor data reporting, heartbeat counting
Write	Master → Slave	0xFFFF2	Link layer confirmation needed	Control command delivery, parameter configuration

In this project, slave sends incrementing count notification via 0xFFFF1 every 6 seconds; master sends fixed message via 0xFFFF2 every 5 seconds. Both operations are asynchronous and independent, non-blocking to each other.

2.4 NimBLE Protocol Stack

ESP32-S3 uses **Apache NimBLE** as BLE Host protocol stack, running in independent FreeRTOS task `vNimbleHostTask`, driving all BLE operations through `nimble_port_run()` event loop. Compared to Bluedroid, NimBLE has smaller code size (~32KB ROM) and lower memory usage (~4.5KB RAM).

3. ESP32-S3 BLE Hardware Features

Feature	Parameter
Wireless standard	BLE 5.0
Max TX power	+20 dBm
RX sensitivity	-97 dBm (1Mbps mode)
Max MTU	251 bytes (this project negotiates 256 upper limit)
Antenna	Onboard PCB antenna
Advertising interval	Protocol stack default (~100ms)
Protocol stack	NimBLE (Apache open source)

BLE and WiFi share 2.4GHz RF front-end, both can coexist via time division.

4. Project Architecture and Flow

```
app_main()
├─ nvs_flash_init()           ← Init NVS flash (stores pairing/bonding info)
├─ ble_slave_init()          ← BLE Slave overall initialization
│   ├─ nimble_port_init()      Init NimBLE protocol stack
│   ├─ ble_hs_cfg callback config      Register sync/reset/register callbacks
│   ├─ ble_svc_gap_device_name_set()   Set advertising device name "ESP-IDF"
│   ├─ ble_svc_gap_init()             GAP service init
│   ├─ ble_svc_gatt_init()            GATT service init
│   ├─ ble_att_set_preferred_mtu(256) Set max transmission unit
│   ├─ xTimerCreate(flush)            Create write buffer flush timer
│   ├─ ble_gatt_register_svcs()       Register GATT service table
│   │   ├─ ble_gatts_count_cfg()      Count validate
│   │   └─ ble_gatts_add_svcs()       Add to protocol stack
│   ├─ xTaskCreate(vNotifyTask)       Create periodic notification task
│   └─ nimble_port_freertos_init()    Start NimBLE host task
│       └─ on_sync()
│           └─ start_advertising() ← Start BLE advertising

gap_event_cb() ← GAP event callback
├─ BLE_GAP_EVENT_CONNECT      Master connected → record handle → MTU negotiate
├─ BLE_GAP_EVENT_MTU         MTU negotiation complete
└─ BLE_GAP_EVENT_DISCONNECT   Disconnected → flush buffer → start_advertising()

gatt_svr_access_cb() ← GATT access callback
    Handles master's write to 0xFFFF2 → buffer → flush timer → print

vNotifyTask() ← Periodic notification task
while(1):
    ├─ vTaskDelay(6000ms)      Every 6 second period
    ├─ sprintf() generate "Hello from ESP32 Slave [N]"
    └─ notify_data() → ble_gattc_notify_custom() Send via 0xFFFF1
```

5. Hardware Dependencies

Hardware Connection: No external wiring, using ESP32-S3 onboard PCB antenna for BLE wireless communication.

Required components: `nvs_flash`, `bt (NimBLE)`, `FreeRTOS`, `esp_log`

Companion device: Need another ESP32-S3 running BLE Master routine as Central to scan and connect to this slave.

6. Source Code Analysis

Full source: `Source Code` → `ESP-IDF` → `5.Network_Protocol` → `9.BLE` → `BLE_Slave`

6.1 Header Files and Macro Configuration

Slave macros are symmetric with Master: two UUIDs (0xFFFF1 notify, 0xFFFF2 write), `NOTIFY_INTERVAL_MS` controls notification interval of 6 seconds, `FLUSH_TIMEOUT_MS` controls write buffer flush timeout of 50ms. `g_connected` marks connection status; notification task silently waits when not connected.

```
#include <stdio.h>
#include <string.h>
#include "esp_log.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/timers.h"
#include "nimble/nimble_port.h"
#include "nimble/nimble_port_freertos.h"
#include "services/gap/ble_svc_gap.h"
#include "services/gatt/ble_svc_gatt.h"
#include "host/ble_hs.h"
#include "host/ble_gap.h"
#include "host/ble_gatt.h"

#define TAG "BLE_SLAVE"
#define DEVICE_NAME "ESP-IDF"
// Service UUID
#define BLE_SVC_UUID 0xFFFF0
// Notify characteristic UUID
#define BLE_NOTIFY_CHR_UUID 0xFFFF1
// Write characteristic UUID
#define BLE_WRITE_CHR_UUID 0xFFFF2
// Notify interval (ms)
#define NOTIFY_INTERVAL_MS 6000
// Flush timeout (ms)
#define FLUSH_TIMEOUT_MS 50

// Global state variables
// Notify val handle
static uint16_t g_notify_val_hdl;
// Write val handle
static uint16_t g_write_val_hdl;
// Write data buffer
static uint8_t g_write_data[512];
// Accumulated len
static uint16_t g_accum_len;
// Current connection handle
static uint16_t g_conn_hdl;
// Connected flag
static bool g_connected;
// Flush timer
static TimerHandle_t g_flush_timer;
```

6.2 app_main() — Entry Point

Init NVS flash to store BLE pairing info, then call `ble_slave_init()` to one-click start slave. If NVS partition abnormal, erase and retry.

```
void app_main(void)
{
    // Init NVS flash
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES ||
        ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        ESP_ERROR_CHECK(nvs_flash_erase());
        ESP_ERROR_CHECK(nvs_flash_init());
    }

    // Start BLE slave
    ble_slave_init();
}
```

6.3 ble_slave_init() — Initialization

`ble_slave_init()` in order: init NimBLE protocol stack → register host callbacks → init GAP/GATT basic services → create write buffer timer → register custom GATT service → create notification task → start host event loop. `ble_gatt_register_svcs()` is a slave-specific step—need to define GATT service table locally for master to discover.

```
void ble_slave_init(void)
{
    int ret;

    // 1. Init NimBLE stack
    ESP_ERROR_CHECK(nimble_port_init());

    // 2. Register callbacks
    ble_hs_cfg.reset_cb = on_reset;
    // Sync callback → start adv
    ble_hs_cfg.sync_cb = on_sync;
    // Register callback → log
    ble_hs_cfg.gatts_register_cb = gatt_svr_register_cb;
    // Store status callback
    ble_hs_cfg.store_status_cb = ble_store_util_status_rr;

    // 3. Init GAP/GATT basic services
    ble_svc_gap_device_name_set(DEVICE_NAME);
    ble_svc_gap_init();
    ble_svc_gatt_init();
    // Set preferred MTU
    ble_att_set_preferred_mtu(256);

    // 4. Create flush timer 50ms
    g_flush_timer = xTimerCreate("flush", pdMS_TO_TICKS(FLUSH_TIMEOUT_MS),
```

```

        pdFALSE, NULL, flush_timer_cb);

// 5. Register custom GATT service
ret = ble_gatt_register_svcs();
if (ret != 0) {
    ESP_LOGE(TAG, "ble_gatt_register_svcs failed; rc=%d", ret);
    return;
}

// 6. Create notification task
ble_gatt_start_notify_task();

// 7. Launch host loop
nimble_port_freertos_init(vNimbleHostTask);
}

```

6.4 on_sync() — Sync Callback and start_advertising()

After protocol stack syncs, `on_sync()` is automatically called, which calls `start_advertising()` internally to start advertising. Advertising contains general discoverable flag, complete device name "ESP-IDF", service UUID 0xFFFF0. `BLE_HS_FOREVER` means continuous advertising without stopping.

```

static void on_sync(void)
{
    ESP_LOGI(TAG, "NimBLE host ready");
    // Start advertising
    start_advertising();
}

void start_advertising(void)
{
    struct ble_gap_adv_params adv_params;
    struct ble_hs_adv_fields fields;
    int rc;

    memset(&fields, 0, sizeof(fields));
    // General discoverable, no BR/EDR
    fields.flags = BLE_HS_ADV_F_DISC_GEN | BLE_HS_ADV_F_BREDR_UNSUP;
    // Device name
    fields.name = (uint8_t *)DEVICE_NAME;
    fields.name_len = strlen(DEVICE_NAME);
    // Complete name
    fields.name_is_complete = 1;
    // Service UUID
    fields.uuids16 = (ble_uuid16_t[]) { BLE_UUID16_INIT(BLE_SVC_UUID) };
    fields.num_uuids16 = 1;
    fields.uuids16_is_complete = 1;

    // Set adv fields
    rc = ble_gap_adv_set_fields(&fields);
    if (rc != 0) {
        ESP_LOGE(TAG, "ble_gap_adv_set_fields failed; rc=%d", rc);
    }
}

```

```

        return;
    }

    uint8_t own_addr_type;
    // Infer addr type
    rc = ble_hs_id_infer_auto(0, &own_addr_type);
    if (rc != 0) {
        ESP_LOGE(TAG, "ble_hs_id_infer_auto failed; rc=%d", rc);
        return;
    }

    memset(&adv_params, 0, sizeof(adv_params));
    // Undirected
    adv_params.conn_mode = BLE_GAP_CONN_MODE_UND;
    // General discovery
    adv_params.disc_mode = BLE_GAP_DISC_MODE_GEN;

    // Continuous advertising
    rc = ble_gap_adv_start(own_addr_type, NULL, BLE_HS_FOREVER,
                          &adv_params, gap_event_cb, NULL);
    if (rc != 0) {
        ESP_LOGE(TAG, "ble_gap_adv_start failed; rc=%d", rc);
        return;
    }

    ESP_LOGI(TAG, "Advertising as \"%s\" ...", DEVICE_NAME);
}

```

6.5 gap_event_cb() — GAP Event Callback

Slave GAP callback handles three events: connection established (record handle, mark connected, negotiate MTU), MTU negotiation complete, disconnection (flush buffer, reset state, automatically re-advertise).

```

int gap_event_cb(struct ble_gap_event *event, void *arg)
{
    switch (event->type) {
        // Connection event
        case BLE_GAP_EVENT_CONNECT:
            if (event->connect.status == 0) {
                // Save conn handle
                g_conn_hdl = event->connect.conn_handle;
                // Set connected
                g_connected = true;
                ESP_LOGI(TAG, "Connected (conn_handle=%d)", g_conn_hdl);
                // Exchange MTU
                ble_gattc_exchange_mtu(g_conn_hdl, NULL, NULL);
            } else {
                ESP_LOGE(TAG, "Connection failed (status=%d)",
                        event->connect.status);
            }
            return 0;
        case BLE_GAP_EVENT_MTU:

```



```

        ESP_LOGI(TAG, "MTU negotiated: %d", event->mtu.value);
        return 0;
    // Disconnect event
    case BLE_GAP_EVENT_DISCONNECT:
        ESP_LOGI(TAG, "Disconnected (reason=%d), restarting advertising...",
            event->disconnect.reason);
        // Flush buffer
        if (g_accum_len > 0) flush_timer_cb(NULL);
        g_conn_hdl = 0;
        // Clear connected
        g_connected = false;
        // Re-advertise
        start_advertising();
        return 0;
    default:
        return 0;
    }
}

```

6.6 GATT Service Definition — Service Table Structure

GATT service defined via C struct array `gatt_svr_svcs[]`: primary service 0xFFFF0 with two characteristics—0xFFFF1 (NOTIFY, slave push) and 0xFFFF2 (WRITE, master write). Each characteristic declares properties via `flags`, `val_handle` is auto-filled by NimBLE after registration, `access_cb` is unified access entry. Service registration first `ble_gatts_count_cfg()` validates table legality, then `ble_gatts_add_svcs()` registers to protocol stack.

```

static const struct ble_gatt_svc_def gatt_svr_svcs[] = {
    {
        // Primary service
        .type = BLE_GATT_SVC_TYPE_PRIMARY,
        .uuid = &((ble_uuid16_t) BLE_UUID16_INIT(BLE_SVC_UUID)).u,
        .characteristics = (struct ble_gatt_chr_def[]) {
            {
                // 0xFFFF1: Notify characteristic
                .uuid = &((ble_uuid16_t)
                    BLE_UUID16_INIT(BLE_NOTIFY_CHR_UUID)).u,
                // Access callback
                .access_cb = gatt_svr_access_cb,
                // Notify property
                .flags = BLE_GATT_CHR_F_NOTIFY,
                // value handle ptr
                .val_handle = &g_notify_val_hdl,
            },
            {
                // 0xFFFF2: write characteristic
                .uuid = &((ble_uuid16_t)
                    BLE_UUID16_INIT(BLE_WRITE_CHR_UUID)).u,
                // Access callback
                .access_cb = gatt_svr_access_cb,
                // write property
                .flags = BLE_GATT_CHR_F_WRITE,
            }
        }
    }
}

```

```

        // value handle ptr
        .val_handle = &g_write_val_hdl,
    },
    // Chr table end
    { 0, }
},
},
// Svc table end
{ 0, }
};

int ble_gatt_register_svcs(void)
{
    // Count validate
    int rc = ble_gatts_count_cfg(gatt_svr_svcs);
    if (rc != 0) {
        ESP_LOGE(TAG, "ble_gatts_count_cfg failed; rc=%d", rc);
        return rc;
    }
    // Add services
    rc = ble_gatts_add_svcs(gatt_svr_svcs);
    if (rc != 0) {
        ESP_LOGE(TAG, "ble_gatts_add_svcs failed; rc=%d", rc);
        return rc;
    }
    return 0;
}

```

6.7 gatt_svr_access_cb() — Write Buffer Accumulation

`gatt_svr_access_cb()` is the slave's only GATT access entry. This project only handles write operations to 0xFFFF2. Each master write appends data to `g_write_data[512]` buffer and resets 50ms timer; if no new data arrives within 50ms, `flush_timer_cb()` prints the complete message at once. Buffer flushes forced when near capacity to prevent overflow.

```

static int gatt_svr_access_cb(uint16_t conn_handle, uint16_t attr_handle,
                             struct ble_gatt_access_ctxt *ctxt, void *arg)
{
    // Not write attr
    if (attr_handle != g_write_val_hdl)
        return BLE_ATT_ERR_UNLIKELY;
    // Not write op
    if (ctxt->op != BLE_GATT_ACCESS_OP_WRITE_CHR) return 0;

    // Write data length
    uint16_t om_len = OS_MBUF_PKTLEN(ctxt->om);
    // About to overflow
    if (g_accum_len + om_len > sizeof(g_write_data) - 1) {
        ESP_LOGW(TAG, "Buffer overflow (%d+%d), flushing",
                 g_accum_len, om_len);
        flush_timer_cb(NULL);
    }
}

```

```

}
uint16_t copied;
// Copy mbuf data to buffer
int rc = ble_hs_mbuf_to_flat(ctxt->om, g_write_data + g_accum_len,
                             sizeof(g_write_data) - g_accum_len - 1,
                             &copied);
if (rc != 0) return BLE_ATT_ERR_UNLIKELY;
// Update accum len
g_accum_len += copied;
// Reset timer
xTimerReset(g_flush_timer, 0);
return 0;
}

void flush_timer_cb(TimerHandle_t timer)
{
    // No data
    if (g_accum_len == 0) return;
    // Add '\0'
    g_write_data[g_accum_len] = '\0';
    // Print complete message
    ESP_LOGI(TAG, "Master(%d bytes): %s", g_accum_len, g_write_data);
    // Reset len
    g_accum_len = 0;
}

```

6.8 notify_data() — Notification Send

Returns directly if not connected or handle invalid. Calls `ble_hs_mbuf_from_flat()` to encapsulate data as NimBLE internal mbuf format, then `ble_gattc_notify_custom()` pushes to master via 0xFFFF1 characteristic. Notification doesn't need master acknowledgment, suitable for periodic sensor data reporting. For reliable transmission, can change characteristic `flags` to `BLE_GATT_CHR_F_INDICATE` (Indicate mode, master confirms after receiving).

```

void notify_data(const char *data, size_t len)
{
    // Guard check
    if (!g_connected || g_notify_val_hdl == 0) return;

    // To mbuf
    struct os_mbuf *om = ble_hs_mbuf_from_flat(data, len);
    if (om == NULL) {
        ESP_LOGE(TAG, "Failed to allocate mbuf for notification");
        return;
    }
    // Send notify
    int rc = ble_gattc_notify_custom(g_conn_hdl, g_notify_val_hdl, om);
    if (rc != 0) {
        ESP_LOGE(TAG, "Notification failed; rc=%d", rc);
    }
}

```

6.9 vNotifyTask() — Periodic Notification Task

`vNotifyTask` is an independent background task running in infinite loop. Every 6 seconds uses `snprintf` to assemble "Hello from ESP32 Slave [N]" (N is incrementing counter), then sends via `notify_data()`. `g_connected` condition ensures when not connected, only silently incrementing counter, no data sent.

```
static void vNotifyTask(void *param)
{
    // Counter
    uint32_t counter = 0;
    char buf[64];
    while (1) {
        // 6s notify interval
        vTaskDelay(pdMS_TO_TICKS(NOTIFY_INTERVAL_MS));
        // Connected and handle valid
        if (g_connected && g_notify_val_hdl != 0) {
            // Build notification message
            int len = snprintf(buf, sizeof(buf),
                               "Hello from ESP32 Slave [%lu]",
                               // Increment counter
                               (unsigned long)counter++);

            // Send notification
            notify_data(buf, len);
            ESP_LOGI(TAG, "Slave: %s", buf);
        }
    }
}
```

`snprintf` uses `unsigned long` type cast for `counter` to ensure on 32-bit platforms `uint32_t` matches `%lu` formatting, avoiding compiler warnings.

7. Operation Flow

7.1 Build and Flash Flow

ESP-IDF Flash Flow: See `ESP-IDF → 1. Before Development → 3. Build and Flash`.

Two ways to test slave: ① Companion another ESP32-S3 flashed with BLE Master routine for dual-device communication (section 7.2); ② Companion phone Bluetooth debugging assistant App for independent testing (section 7.3).

7.2 Testing Method 1: With ESP32 Master

1. Power on Slave board, serial outputs `Advertising as "ESP-IDF" ...` indicating advertising started.
2. Power on Master board, Master scans and automatically connects to `ESP-IDF`.
3. Slave serial outputs `Connected (conn_handle=1)` and `MTU negotiated: 256`, indicating connection established.
4. Wait ~6 seconds, Slave serial outputs `Slave: Hello from ESP32 Slave [0]`, indicating notification sent successfully.

5. Wait ~5 seconds, Slave serial outputs `Master(23 bytes): Hello from ESP32 Master`, indicating received master write data.
6. Both message types output alternately and independently, proving bidirectional data channel works.

```
I (472) BLE_SLAVE: NimBLE host ready
I (472) NimBLE: GAP procedure initiated: advertise;
I (482) NimBLE: disc_mode=2
I (482) NimBLE: adv_channel_map=0 own_addr_type=0 adv_filter_policy=0 adv_itvl_min=0 adv_itvl_max=0
I (492) NimBLE:

I (492) BLE_SLAVE: Advertising as "ESP-IDF" ...
I (492) main_task: Returned from app_main()
I (2672) BLE_SLAVE: Connected (conn_handle=1)
I (2682) NimBLE: GATT procedure initiated: exchange mtu

I (2732) BLE_SLAVE: MTU negotiated: 256
I (2782) BLE_SLAVE: MTU negotiated: 256
I (3272) NimBLE: ATT write not permitted; conn_handle=0x0001 attr_handle=0x0010

I (3622) BLE_SLAVE: Master(23 bytes): Hello from ESP32 Master
I (6432) NimBLE: GATT procedure initiated: notify;
I (6432) NimBLE: att_handle=16

I (6432) BLE_SLAVE: Slave: Hello from ESP32 Slave [0]
I (8622) BLE_SLAVE: Master(23 bytes): Hello from ESP32 Master
I (12432) NimBLE: GATT procedure initiated: notify;
I (12432) NimBLE: att_handle=16

I (12432) BLE_SLAVE: Slave: Hello from ESP32 Slave [1]
I (13622) BLE_SLAVE: Master(23 bytes): Hello from ESP32 Master
I (18432) NimBLE: GATT procedure initiated: notify;
I (18432) NimBLE: att_handle=16
```

7.3 Testing Method 2: With Phone Bluetooth Debugging Assistant

Besides ESP32 dual-device testing, can also use phone Bluetooth debugging assistant App to connect to slave and read/write data. Suitable for having only one development board or wanting quick verification of BLE communication.

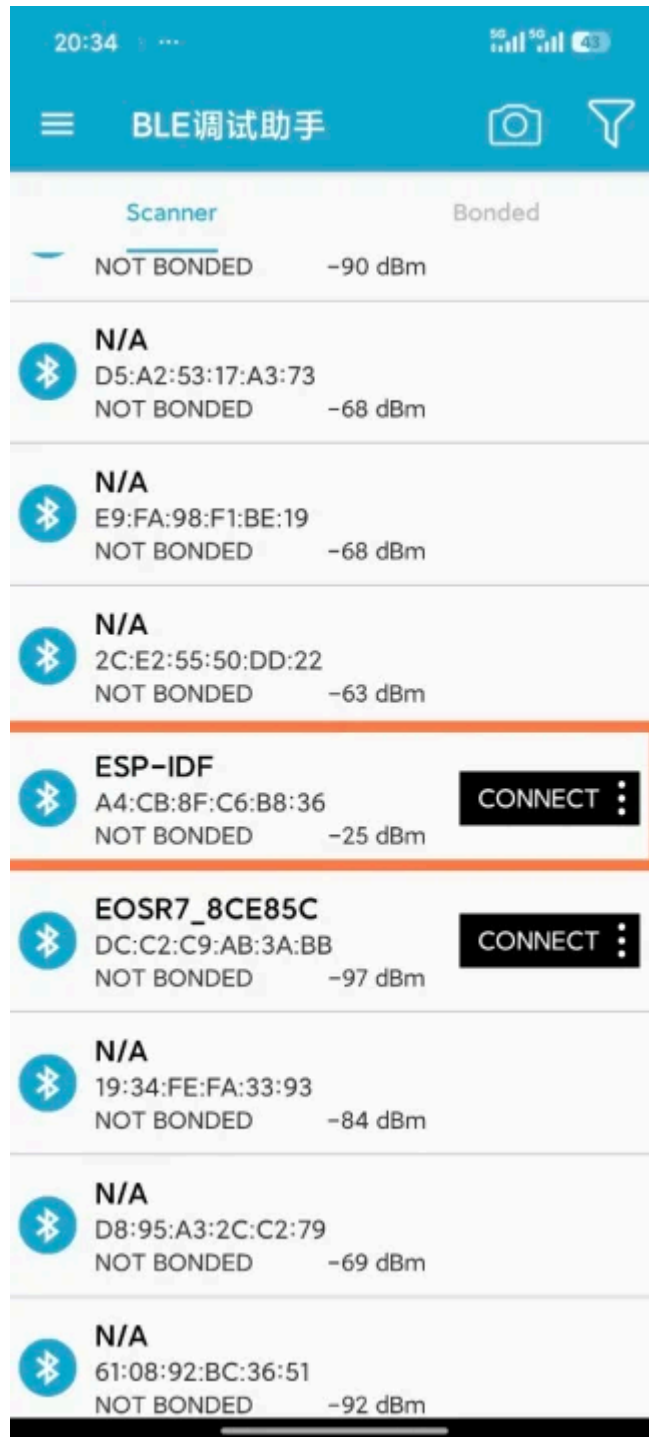
Common Apps: nRF Connect for Mobile (recommended, most full-featured), LightBlue, EFR Connect, BLE Scanner, all free.

Steps:

1. Power on Slave board, serial outputs `Advertising as "ESP-IDF" ...` indicating advertising started.

```
I (527) BLE_SLAVE: Advertising as "ESP-IDF" ...
I (527) main_task: Returned from app_main()
```

2. Open phone Bluetooth debugging assistant App, scan for nearby devices, find device named `ESP-IDF`.



3. Click connect. Slave serial outputs `Connected` and `MTU negotiated: 256`.

4. In App, find service `0xFFFD`, expand to see two characteristics:

- **0xFFFD1 (NOTIFY)**: Click "Enable Notify"/"Start Notification", thereafter every 6 seconds App receives a push of `Hello from ESP32 slave [N]`.

20:35

5G 5G 43



ESP-IDF

断开连接



Services

Log

Generic Access

> UUID: 00001800-0000-1000-8000-00805f9b34fb
PRIMARY SERVICE

Generic Attribute

> UUID: 00001800-0000-1000-8000-00805f9b34fb

接收数据

☒ 实时显示 ☐ 保存至文件

接收:52 Byte

HEX ☐

实时速度:26 B/s

接收通知数据 ☒

Hello from ESP32 Slave [7]Hello from
ESP32 Slave [8]

关闭

清空

0805f9b34fb

Properties:WRITE



- **0xFFFF2 (WRITE):** Click "Write", enter any text (e.g. "Hello from Phone") and send. Slave serial prints `Master(N bytes): Hello from Phone`.

```
I (258467) BLE_SLAVE: Slave: Hello from ESP32 Slave [3]
I (259227) BLE_SLAVE: Master(16 bytes): Hello from phone
I (264467) NimBLE: GATT procedure initiated: notify;
I (264467) NimBLE: att_handle=16
```

Slave serial output (phone connected):

```
I (452) BLE_SLAVE: NimBLE host ready
I (492) BLE_SLAVE: Advertising as "ESP-IDF" ...
I (492) main_task: Returned from app_main()
I (2672) BLE_SLAVE: Connected (conn_handle=1)
I (2732) BLE_SLAVE: MTU negotiated: 256
I (6432) BLE_SLAVE: Slave: Hello from ESP32 Slave [0]
I (8000) BLE_SLAVE: Master(18 bytes): Hello from Phone
I (12432) BLE_SLAVE: Slave: Hello from ESP32 Slave [1]
```

In nRF Connect, after enabling Notify, characteristic value area continuously updates notification content; Write operation requires manually entering data and clicking send.

7.4 Normal Communication Log (Dual-Board Test)

Field	Example	Meaning
<code>I</code>	<code>I</code>	INFO level log
<code>BLE_SLAVE</code>	<code>BLE_SLAVE</code>	Log tag
<code>Slave:</code>	<code>Slave:</code>	Notification proactively sent by slave
<code>Master(N bytes):</code>	<code>Master:</code>	Data from master write
<code>Hello from ESP32 Slave [0]</code>	Notification content	Incrementing count message


```

I (452) BLE_SLAVE: NimBLE host ready
I (492) BLE_SLAVE: Advertising as "ESP-IDF" ...
I (492) main_task: Returned from app_main()
I (2672) BLE_SLAVE: Connected (conn_handle=1)
I (2732) BLE_SLAVE: MTU negotiated: 256
I (3272) NimBLE: ATT write not permitted; conn_handle=1 attr_handle=0x0010
I (3622) BLE_SLAVE: Master(23 bytes): Hello from ESP32 Master
I (6432) BLE_SLAVE: Slave: Hello from ESP32 Slave [0]
I (8622) BLE_SLAVE: Master(23 bytes): Hello from ESP32 Master
I (12432) BLE_SLAVE: Slave: Hello from ESP32 Slave [1]
I (13622) BLE_SLAVE: Master(23 bytes): Hello from ESP32 Master

```

```

I (472) BLE_SLAVE: NimBLE host ready
I (472) NimBLE: GAP procedure initiated: advertise;
I (482) NimBLE: disc_mode=2
I (482) NimBLE: adv_channel_map=0 own_addr_type=0 adv_filter_policy=0 adv_itvl_min=0 adv_itvl_max=0
I (492) NimBLE:
I (492) BLE_SLAVE: Advertising as "ESP-IDF" ...
I (492) main_task: Returned from app_main()
I (2672) BLE_SLAVE: Connected (conn_handle=1)
I (2682) NimBLE: GATT procedure initiated: exchange mtu
I (2732) BLE_SLAVE: MTU negotiated: 256
I (2782) BLE_SLAVE: MTU negotiated: 256
I (3272) NimBLE: ATT write not permitted; conn_handle=0x0001 attr_handle=0x0010
I (3622) BLE_SLAVE: Master(23 bytes): Hello from ESP32 Master
I (6432) NimBLE: GATT procedure initiated: notify;
I (6432) NimBLE: att_handle=16
I (6432) BLE_SLAVE: Slave: Hello from ESP32 Slave [0]
I (8622) BLE_SLAVE: Master(23 bytes): Hello from ESP32 Master
I (12432) NimBLE: GATT procedure initiated: notify;
I (12432) NimBLE: att_handle=16
I (12432) BLE_SLAVE: Slave: Hello from ESP32 Slave [1]
I (13622) BLE_SLAVE: Master(23 bytes): Hello from ESP32 Master
I (18432) NimBLE: GATT procedure initiated: notify;
I (18432) NimBLE: att_handle=16

```

About `ATT write not permitted`: This NimBLE underlying error appears on the 3rd line of slave log. `attr_handle=0x0010` is the value handle of `0xFFFF1` notify characteristic—this characteristic only declares `BLE_GATT_CHR_F_NOTIFY` property, not allowing external writes. **When master enables CCCD during GATT discovery phase, should actually write to CCCD descriptor handle, but NimBLE protocol stack in callback chain first tries to do a write verification on the notify characteristic value handle; slave rejects based on property, then protocol stack internally corrects routing to the correct descriptor handle, not affecting subsequent functionality.** Subsequent `Master(23 bytes)` goes through `0xFFFF2` write characteristic handle (property includes `BLE_GATT_CHR_F_WRITE`), received normally.

7.5 Edge Cases

- **Slave runs standalone (not connected):** Continuously advertises, `g_connected=false`, notification task silently increments counter without sending data.
- **Master disconnects:** Slave receives `BLE_GAP_EVENT_DISCONNECT` then automatically resets state and re-advertises, no reset needed.
- **Write buffer not flushed when Master disconnects:** `gap_event_cb()` actively calls `flush_timer_cb(NULL)` during disconnection handling to clear, avoiding residual data from previous connection.
- **Master writes rapidly causing buffer overflow:** `gatt_svr_access_cb()` checks capacity before appending; if about to overflow, forces flush then receives new data.

8. Common Problems

Symptom	Cause	Solution
Bluetooth init fails	NVS partition corrupted or insufficient space	IDE flash with "Erase flash" checked, or manually <code>idf.py erase-flash</code>
Slave advertises but Master can't find it	Device name mismatch / advertising didn't start successfully	Confirm <code>DEVICE_NAME</code> is "ESP-IDF", Master side <code>TARGET_DEVICE_NAME</code> matches
Slave notification send fails	Not connected / <code>g_notify_val_hdl</code> is 0	Confirm GATT service registered successfully, characteristic flags include <code>BLE_GATT_CHR_F_NOTIFY</code>
Master writes but Slave doesn't receive	<code>g_write_val_hdl</code> is 0 / CCCD not configured	Confirm 0xFF2 characteristic flags include <code>BLE_GATT_CHR_F_WRITE</code>
Disconnects immediately after connection	MTU negotiation failed / connection parameters incompatible	Ensure two devices are within 5 meters, no strong electromagnetic interference
<code>ble_gatts_count_cfg</code> returns error	Service table structure definition illegal (UUID/NULL terminator missing)	Confirm service table ends with <code>{0,}</code> , characteristic table ends with <code>{0,}</code>
Write buffer accumulation doesn't print	Timer not started / timeout too long	Confirm <code>g_flush_timer</code> created successfully and <code>FLUSH_TIMEOUT_MS=50</code>
Advertising start fails <code>rc=12</code>	BLE controller not enabled	menuconfig ensure <code>CONFIG_BT_ENABLED=y</code> , <code>CONFIG_BTDM_CTRL_MODE_BLE_ONLY=y</code>